

CuidaMais

Documentação Técnica do Sistema

Back-end · Banco de Dados · APIs · Regras de Negócio

Versão 1.0 — 2025

1. Visão Geral do Sistema

O CuidaMais é uma aplicação web de monitoramento de saúde domiciliar voltada para cuidadores de idosos. O sistema permite registrar sinais vitais, acompanhar o histórico clínico, visualizar alertas de risco e gerar relatórios médicos.

A aplicação é construída com Next.js 14 (App Router), utilizando Prisma ORM com banco de dados SQLite e autenticação baseada em JWT armazenado em cookie HttpOnly.

1.1 Stack Tecnológica

Camada	Tecnologia
Framework	Next.js 14 — App Router com React Server Components
ORM / Banco	Prisma ORM + SQLite (arquivo local database.db)
Autenticação	JWT (jose) + bcryptjs para hash de senha + cookie session
API Routes	Next.js Route Handlers (App Router) em /app/api/
Front-end	React + Tailwind CSS + Lucide Icons

2. Banco de Dados e Schema

O banco de dados é SQLite gerenciado pelo Prisma ORM. O schema define três modelos com relações em cascata: User → Patient → Record.

2.1 Diagrama de Relacionamento (ERD)

Estrutura de Relacionamento

User (1) —→ (N) Patient (1) —→ (N) Record Um usuário cadastrado pode ter vários pacientes. Cada paciente acumula múltiplos registros de plantão ao longo do tempo. A deleção em cascata garante que excluir um usuário remove seus pacientes, e excluir um paciente remove todos os seus registros.

2.2 Tabela de Modelos

Modelo	Campos principais	Descrição
User	id, email, password, name, role, createdAt	Armazena os cuidadores/usuários do sistema. Relação 1:N com Patient.
Patient	id, name, age, status, condition, ownerId, createdAt	Representa um paciente monitorado. Vinculado a um User via ownerId (FK).
Record	id, patientId, createdAt + 30+ campos clínicos	Registro de plantão. Cada entrada é um snapshot completo do estado do paciente.

2.3 Modelo User

Representa o cuidador que usa a plataforma. O campo role tem valor padrão "admin" (para uso futuro com múltiplos perfis de acesso). A senha nunca é armazenada em texto plano — é sempre hashada com bcrypt antes de persistir no banco.

Campo	Tipo	Obrigatório	Observação
id	String (UUID)	Sim	Gerado automaticamente
email	String	Sim	Único no banco (constraint unique)
password	String	Sim	Hash bcrypt (salt 10)
name	String	Sim	Nome do cuidador
role	String	Não	Padrão: "admin"

2.4 Modelo Patient

Cada paciente pertence a exatamente um cuidador (ownerId é a FK para User). O campo status indica se o paciente está em monitoramento ativo. A relação onDelete: Cascade garante limpeza automática dos dados ao remover um usuário.

Campo	Tipo	Obrigatório	Observação
id	String (CUID)	Sim	Gerado automaticamente
name	String	Sim	Nome do paciente
age	Int	Sim	Idade em anos
status	String	Não	Padrão: "Active"
condition	String?	Não	Diagnóstico/condição médica
ownerId	String	Sim	FK para User.id — isolamento de dados por cuidador

2.5 Modelo Record — Campos Clínicos Completos

O Record é o coração do sistema. Cada entrada representa um registro de plantão e cobre 8 domínios clínicos com mais de 30 campos. Todos os campos numéricos/textuais são opcionais (nullable), enquanto os booleanos possuem valores padrão para garantir integridade.

Grupo	Campos	Tipos / Descrição
Sinais Vitais	systolic, diastolic, glucose, heart_rate, oxygen, temperature	Int?/Float? — valores numéricos opcionais
Sintomas Físicos	weight, pain_level, pain_location, fatigue, dizziness, edema	Float?/Int?/Boolean — queixas físicas
Mobilidade	mobility, recent_falls, difficulty_standing, support_equipment	String/Boolean — capacidade de locomoção
Estado Mental	oriented, mental_confusion, excessive_sleepiness, speech_alteration	Boolean — cognição e consciência
Nutrição	appetite, food_intake, water_intake, difficulty_swallowing	String/Boolean — ingestão alimentar
Eliminação	urine, feces, incontinence	String — função excretora
Bem-estar	mood, activity_interest, sleep_quality	String/Boolean — estado emocional
Observações	notes	String? — texto livre do cuidador

Nota sobre IDs

O modelo Record utiliza @default(uuid()) diferente dos outros modelos que usam @default(cuid()). Ambos geram IDs únicos, mas UUID é o padrão mais universal para integrações externas.

3. Sistema de Autenticação

A autenticação é implementada sem bibliotecas de sessão externas (como NextAuth), utilizando apenas JWT via jose e hashing via bcryptjs. O fluxo é completamente stateless no servidor — o estado de sessão vive no token.

3.1 Fluxo de Cadastro (Sign Up)

- Cliente envia POST /api/auth/signup com { name, email, password }
- Servidor chama hashPassword() — bcrypt.hash(password, 10)
- Prisma cria o registro User com a senha hasheada
- Retorna status 201 com os dados do usuário (sem a senha)

3.2 Fluxo de Login (Sign In)

- Cliente envia POST /api/auth/signin com { email, password }
- Servidor busca o usuário pelo email no banco
- Chama comparePassword() — bcrypt.compare(password, hash) para validar
- Em caso de sucesso, chama signToken({ userId, email }) para gerar JWT com validade de 7 dias
- JWT é armazenado em cookie HttpOnly chamado "session" com Set-Cookie
- Redireciona o cliente para /dashboard

3.3 Proteção de Rotas — requireSession()

Toda rota autenticada começa chamando requireSession(). A função:

- Lê o cookie session do request via next/headers
- Chama verifyToken() que usa jwtVerify() do jose com a chave secreta JWT_SECRET
- Retorna o payload { userId, email } se o token for válido
- Lança erro "Não autenticado." se o token estiver ausente ou inválido

As rotas capturam esse erro e retornam HTTP 401 automaticamente.

Segurança: Isolamento de Dados por cuidador

Toda consulta ao banco que envolve pacientes ou registros sempre filtra por ownerId: session.userId. Isso garante que um cuidador nunca acesse dados de outro, mesmo que tente adivinhar um ID válido.

3.4 Configuração do Segredo JWT

O segredo é lido de process.env.JWT_SECRET. Se a variável não estiver definida, usa um fallback hard-coded apenas para desenvolvimento. Em produção, é obrigatório definir JWT_SECRET no ambiente.

```
const SECRET = new TextEncoder().encode(  
  process.env.JWT_SECRET ?? "segredo-temporario-projeto30-12345"  
);
```

4. Rotas da API (Route Handlers)

Todas as rotas estão em `/app/api/` e seguem o padrão de Route Handlers do Next.js App Router. A instância Prisma é compartilhada globalmente para evitar múltiplas conexões em desenvolvimento (hot reload).

4.1 Mapa Completo de Endpoints

Método	Rota	Função	Acesso
POST	<code>/api/auth/signup</code>	Cria novo usuário (hash de senha com bcrypt)	Público
POST	<code>/api/auth/signin</code>	Autentica, cria JWT e seta cookie session	Público
GET	<code>/api/patients</code>	Lista pacientes do usuário autenticado	Autenticado
POST	<code>/api/patients</code>	Cria novo paciente vinculado ao usuário	Autenticado
GET	<code>/api/records?patientId=X</code>	Busca o registro mais recente do paciente	Autenticado
POST	<code>/api/records</code>	Salva novo registro clínico completo	Autenticado
GET	<code>/api/history</code>	Retorna todos os registros do primeiro paciente	Autenticado
GET	<code>/api/stats</code>	Retorna dados formatados para gráficos (15 últimos registros)	Autenticado
GET	<code>/api/alerts?patientId=X</code>	Gera alertas clínicos baseados no último registro	Autenticado
GET	<code>/api/report</code>	Gera resumo estatístico para relatório médico	Autenticado
GET	<code>/paciente/[id]</code>	Visualização pública do histórico do paciente	Público

4.2 API de Pacientes — /api/patients

POST — Criar Paciente

- Autenticação obrigatória via requireSession()
- Recebe { name, age, condition } no body
- name e age são validados como obrigatórios (400 se ausentes)
- ownerId é injetado automaticamente a partir da sessão — o cliente nunca envia isso
- Status inicial é sempre "Active"
- Retorna o objeto Patient criado com status 201

GET — Listar Pacientes

- Filtra por ownerId: session.userId para isolamento completo
- Inclui o último registro de cada paciente (records com orderBy + take: 1)
- Retorna array de pacientes ordenados por data de criação decrescente

4.3 API de Registros — /api/records

POST — Salvar Registro Clínico

- Valida que patientId foi enviado
- Verifica que o paciente existe E pertence ao cuidador autenticado (segurança dupla)
- Faz parsing de todos os campos: parseInt() para inteiros, parseFloat() para decimais
- Booleanos são normalizados com !!valor (converte truthy/falsy para boolean)
- Campos não enviados ficam null (sem valor padrão forçado no POST)
- Retorna o Record criado com status 201

GET — Buscar Último Registro

- Requer patientId como query param
- Verifica propriedade do paciente via patient: { ownerId: session.userId }
- Retorna o registro mais recente (orderBy createdAt desc, primeiro resultado)
- Retorna array vazio [] se não houver registros

4.4 API de Alertas — /api/alerts

Esta rota implementa a lógica de monitoramento clínico automatizado. Busca o registro mais recente do paciente e avalia cada indicador contra thresholds médicos predefinidos.

Thresholds e Alertas Gerados

Condição avaliada	Severidade	Alerta gerado
sistólica >= 140 mmHg	critical	Hipertensão Detectada
sistólica <= 90 mmHg	warning	Hipotensão Detectada
glicemia >= 180 mg/dL	critical	Hiperglicemia Severa
glicemia <= 70 mg/dL	critical	Hipoglicemia (Emergência)
temperatura >= 37.8 °C	critical	Estado Febril / Hipertermia
saturação < 94%	critical	Baixa Saturação de Oxigênio (SpO ₂)
recent_falls = true	critical	Queda nas últimas 24h
mental_confusion = true	warning	Sinal de Confusão Mental

Lógica de Avaliação

Os alertas são gerados a partir de uma única leitura — o último registro do paciente. Não há média histórica nesta versão. Cada alerta retorna { id, type, title, description } onde type é "critical" ou "warning".

4.5 API de Estatísticas — /api/stats

Retorna os últimos 15 registros formatados para consumo por componentes de gráfico (Recharts). Os dados são ordenados de forma crescente por data para exibição cronológica correta nos eixos.

Campos retornados por ponto:

- data — data formatada em pt-BR (dd/MM)
- systolic, diastolic — pressão arterial
- glucose — glicemia em mg/dL
- temperature — temperatura em °C
- heart_rate — frequência cardíaca

4.6 API de Relatório Médico — /api/report

Consolida os últimos 30 registros em um resumo estatístico para compartilhar com médicos ou familiares. Calcula:

- Médias de pressão sistólica/diastólica, glicemia e temperatura
- Contagem total de intercorrências: quedas, confusão mental, edema
- Últimas 5 notas do cuidador com data
- Retorna { empty: true } quando não há registros disponíveis

5. Gerenciamento da Conexão com o Banco

Em Next.js com hot-reload ativo em desenvolvimento, cada alteração de arquivo re-executa os módulos, o que poderia criar dezenas de conexões com o banco. O arquivo `prisma.ts` resolve isso com o padrão Singleton via global:

```
const globalForPrisma = global as unknown as { prisma: PrismaClient };

export const prisma = globalForPrisma.prisma || new PrismaClient();

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

Em produção (`NODE_ENV = 'production'`), cada instância do servidor cria seu próprio `PrismaClient` normalmente, pois não há hot-reload. Em desenvolvimento, a instância é salva no objeto global do Node.js e reutilizada entre reloads.

6. Visualização Pública do Paciente

A rota `/paciente/[id]` é uma Server Component pública (sem autenticação) que permite compartilhar o histórico de um paciente com médicos ou familiares via link direto.

6.1 Comportamento

- Acessa o banco diretamente via `prisma.patient.findUnique()` — sem camada de API
- Inclui todos os registros ordenados por data decrescente (todos, sem limite)
- Retorna `notFound()` (404) se o ID não existir
- Exibe sinais vitais, dados clínicos, notas e timestamps de cada registro

Atenção: Rota Pública sem Autenticação

Qualquer pessoa com o link `/paciente/[id]` pode ver o histórico completo. O ID é um UUID gerado automaticamente (difícil de adivinhar por força bruta), mas considere adicionar proteção adicional (token de compartilhamento ou expiração) para dados sensíveis.

7. Fluxo Completo de Dados — Do Cadastro ao Alerta

Passo	Ação	O que acontece no back-end
1	Cadastro do cuidador	POST /api/auth/signup → bcrypt.hash() → prisma.user.create()
2	Login	POST /api/auth/signin → bcrypt.compare() → signToken() → cookie session
3	Criação do paciente	POST /api/patients → requireSession() → prisma.patient.create({ ownerId })
4	Registro de plantão	POST /api/records → validação de propriedade → prisma.record.create() com 30+ campos
5	Visualização de alertas	GET /api/alerts?patientId=X → último Record → avaliação de thresholds → array de alertas
6	Gráficos / Dashboard	GET /api/stats → 15 últimos Records → formatação para Recharts
7	Relatório médico	GET /api/report → 30 últimos Records → médias + contagem de intercorrências
8	Compartilhamento	GET /paciente/[id] → Server Component pública → todos os Records

8. Sistema de Notificações (Toast)

O sistema de toasts é implementado sem biblioteca externa, usando o Event System nativo do browser com CustomEvent. Funciona como um barramento de eventos desacoplado.

8.1 Funcionamento

- `toast(kind, title, description?)` dispara um CustomEvent "app:toast" com os dados da mensagem
- `useToasts()` é um hook que escuta esse evento e mantém o array de mensagens ativas em estado React
- Cada mensagem tem um ID numérico sequencial e expira automaticamente após 4.5 segundos via `setTimeout`
- Tipos suportados: success, error, info

Vantagem da abordagem por eventos

Como `toast()` é uma função simples que dispara um evento global, pode ser chamada de qualquer lugar da aplicação — componentes de servidor não têm acesso, mas qualquer Client Component ou utilitário pode emitir notificações sem prop drilling ou context providers.

9. Considerações de Segurança e Melhorias Futuras

9.1 Pontos de Atenção Atuais

- A variável `JWT_SECRET` tem fallback hard-coded — em produção deve ser sempre configurada via variável de ambiente
- A rota `/paciente/[id]` não tem autenticação — recomenda-se adicionar token de compartilhamento com expiração
- O banco SQLite é adequado para uso doméstico/pequeno, mas para escala maior considere PostgreSQL ou MySQL
- Não há rate limiting nas rotas de autenticação — recomendado adicionar para prevenir força bruta
- A lógica de alertas avalia apenas o último registro — considere avaliar tendências temporais para alertas mais precisos

9.2 Melhorias Sugeridas

- Migrar para PostgreSQL via Prisma (apenas mudar datasource provider e url)
- Adicionar middleware de autenticação no Next.js para redirecionar automaticamente usuários não autenticados
- Implementar paginação na API de histórico para pacientes com muitos registros
- Adicionar webhooks ou push notifications para alertas críticos em tempo real
- Implementar soft delete para manter histórico mesmo após exclusão de pacientes
- Criar role-based access para distinguir cuidadores, supervisores e médicos

10. Front-end — Estrutura de Páginas e Componentes

O front-end é inteiramente construído com React Client Components dentro do Next.js App Router. O design system utiliza Tailwind CSS com tokens personalizados (oklch color space) e ícones do Lucide React. Abaixo estão todas as páginas e seus comportamentos.

10.1 Mapa de Rotas do Front-end

Rota	Arquivo	Descrição
/	app/page.tsx	Landing page pública (não enviada, presumida)
/auth	app/auth/page.tsx	Página unificada de login e cadastro
/dashboard	app/dashboard/page.tsx	Painel principal com sinais vitais do último registro
/dashboard/registro	app/dashboard/registro/page.tsx	Formulário completo de registro diário clínico
/dashboard/historico	app/dashboard/historico/page.tsx	Tabela cronológica com todos os registros do paciente
/dashboard/graficos	app/dashboard/graficos/page.tsx	4 gráficos de linha com evolução dos sinais vitais
/dashboard/alertas	app/dashboard/alertas/page.tsx	Central de alertas clínicos automáticos
/dashboard/relatorio	app/dashboard/relatorio/page.tsx	Relatório médico consolidado com suporte a impressão
/dashboard/pacientes	app/dashboard/pacientes/page.tsx	Gerenciamento e seleção de pacientes monitorados
/share/[id]	app/share/[id]/page.tsx	Visualização pública do prontuário (Server Component)

10.2 Layout do Dashboard — DashboardLayout

Todas as páginas dentro de /dashboard compartilham um layout com sidebar fixa. O layout é um Client Component que envolve tudo com o PatientProvider, injetando o contexto de paciente selecionado em toda a árvore de componentes filhos.

Estrutura visual

- Sidebar fixa à esquerda (264px) com logo, navegação e botão de logout
- Banner azul na sidebar indicando qual paciente está em monitoramento
- Área de conteúdo principal ocupa o restante da tela (flex-1, pl-64)
- Sidebar e padding são zerados automaticamente na impressão (print:hidden / print:pl-0)

Logout

O botão de sair chama POST /api/auth/signout no servidor (que limpa o cookie session) e redireciona o usuário para /auth via useRouter().push().

10.3 Página de Autenticação — /auth

Página unificada que alterna entre dois modos via estado local: "signin" (entrar) e "signup" (criar conta). Usa um toggle de botões para alternar entre os formulários sem trocar de rota.

Modo	Comportamento
signup	Exibe campo de nome + email + senha. POST /api/auth/signup. Em sucesso, troca para modo signin.
signin	Email + senha. POST /api/auth/signin. Em sucesso, router.push('/dashboard').

O layout é dividido em duas colunas em telas grandes: lado esquerdo com gradiente azul/verde e slogan, lado direito com o formulário. Em mobile, apenas o formulário é exibido.

10.4 Dashboard Principal — /dashboard

Exibe um resumo do estado atual do paciente selecionado. Consome a rota GET `/api/dashboard/records/patients/[id]` para buscar os registros e pega o mais recente (`records[0]`).

Componente Stat

Componente reutilizável que renderiza um card com ícone colorido, label, valor e unidade. Suporta 5 "tons" de cor via prop `tone`: `primary` (azul), `success` (verde), `warning` (âmbar), `info` (ciano), `destructive` (rosa).

Cards exibidos

Card	Campo do Record	Unidade	Cor
Pressão	systolic/diastolic	mmHg	Azul (primary)
Glicemia	glucose	mg/dL	Verde (success)
Temperatura	temperature	°C	Âmbar (warning)
Frequência	heart_rate	bpm	Rosa (destructive)
Saturação	oxygen	%	Ciano (info)
Humor	mood	—	Verde (success)

Abaixo dos cards de vitais, há 3 painéis secundários: Mobilidade (`mobility + support_equipment`), Nutrição (`appetite + water_intake`) e Estado Mental (`oriented`).

10.5 Página de Pacientes — /dashboard/pacientes

Central de gerenciamento dos pacientes vinculados ao cuidador. É também o ponto de entrada obrigatório para ativar o monitoramento — sem um paciente selecionado, todas as outras páginas exibem um estado vazio.

Funcionalidades

- Listagem em grid (1 col mobile, 2 col tablet, 3 col desktop) com cards por paciente
- Card mostra iniciais, nome, idade, condição clínica e status de seleção
- Clique no card chama `selectPatient()` do contexto, salvando no `localStorage` e atualizando a sidebar
- Botão de copiar link gera a URL `/share/[id]` e copia para o clipboard
- Botão de excluir chama `DELETE /api/dashboard/records/patients/[id]` com confirmação
- Modal de criação com campos: Nome, Idade, Condições Clínicas

Auto-seleção do primeiro paciente

Ao criar o primeiro paciente (lista estava vazia), o sistema o seleciona automaticamente via `selectPatient()` após a resposta da API, evitando que o cuidador precise de um passo extra para começar a monitorar.

10.6 Página de Histórico — /dashboard/historico

Tabela cronológica completa de todos os registros do paciente selecionado. Consome GET /api/dashboard/records/history?patientId=X.

Colunas da tabela

Coluna	Campo fonte	Formatação
Data / Hora	record.createdAt	toLocaleDateString pt-BR com hora e minuto
P.A.	systolic + diastolic	"120/80" ou "—" se nulo
Glicemia	glucose	valor + "mg/dL" ou "—"
Temp.	temperature	valor + "°C" ou "—"
Freq. Cardíaca	heart_rate	valor + "bpm" ou "—"
SpO ₂	oxygen	valor + "%" ou "—"
Humor	mood	badge arredondado cinza
Evolução / Notas	notes	texto truncado com tooltip no hover

10.7 Página de Gráficos — /dashboard/graficos

Renderiza 4 gráficos de linha usando Recharts com os últimos 15 registros do paciente (GET /api/dashboard/records/stats?patientId=X). Os gráficos são renderizados somente após a hidratação do componente (hasMounted) para evitar erros de SSR.

Comportamento responsivo

Um listener de resize calcula dinamicamente a largura dos gráficos: mobile (largura da tela menos 48px), tablet (menos 80px) e desktop (520px fixo). Isso evita overflow horizontal em telas pequenas.

Gráficos renderizados

Gráfico	Campos plotados	Cor das linhas
Pressão Arterial	systolic, diastolic	Azul #2563eb (sistólica), Ciano #38bdf8 (diastólica)
Glicemia Capilar	glucose	Verde esmeralda #10b981
Temperatura Corporal	temperature	Âmbar #f59e0b — eixo Y fixo entre 35°C e 40°C
Frequência Cardíaca	heart_rate	Rosa/vermelho #e11d48

Todos os gráficos usam connectNulls={true} para conectar pontos mesmo quando há campos ausentes em algum registro, evitando quebras na linha.

10.8 Página de Alertas — /dashboard/alertas

Consome GET /api/dashboard/records/alerts?patientId=X e renderiza os alertas retornados. Reage reativamente ao paciente selecionado no contexto via useEffect([selectedPatient?.id]).

Estados de renderização

- Sem paciente selecionado: card vazio com instrução para ir à aba Pacientes
- Carregando: placeholder com texto "Processando triagem de sinais vitais..."
- Sem alertas: card verde com ShieldCheck — "Nenhum Alerta Crítico Ativo"
- Com alertas: banner vermelho com contador + listagem de cards coloridos

Visual dos alertas

Cada card de alerta tem cor dinâmica: vermelho (rose) para type="critical" e âmbar para type="warning". O ícone AlertTriangle é compartilhado, mas o fundo e texto mudam de acordo com a severidade.

10.9 Página de Relatório Médico — /dashboard/relatorio

Página otimizada para impressão. Consome GET /api/dashboard/records/report (sem patientId — usa o primeiro paciente do cuidador logado). Estrutura o laudo em 3 blocos formais.

Blocos do relatório

- Cabeçalho do laudo com nome do sistema, data de emissão e período amostral
- Bloco 1 — Médias Clínicas: cards com média de P.A., glicemia e temperatura
- Bloco 2 — Sumário de Eventos: contadores de quedas, confusão mental e edema (vermelho/âmbar quando > 0)
- Bloco 3 — Evoluções Narrativas: até 5 notas do cuidador com data
- Espaço para assinatura e carimbo do médico

Suporte a Impressão (CSS @media print)

A sidebar, o botão de impressão e bordas decorativas recebem a classe print:hidden ou print:border-0. O conteúdo do laudo expande para ocupar a largura total da folha com print:absolute print:top-0 print:left-0 print:w-full, produzindo um PDF limpo ao usar Ctrl+P ou o botão "Imprimir / Salvar PDF".

11. Gerenciamento de Estado Global

O CuidaMais não usa Redux, Zustand ou qualquer biblioteca de estado global externa. O estado compartilhado entre páginas é gerenciado por dois mecanismos complementares.

11.1 PatientContext — Contexto React

O arquivo PatientContext.tsx define um Context API com Provider e hook personalizado. É o principal mecanismo de estado global da aplicação.

Export	Descrição
PatientProvider	Wrapper que inicializa o estado. Lê localStorage na montagem (useEffect) para restaurar o paciente entre sessões.
usePatient()	Hook que expõe { selectedPatient, selectPatient }. Lança erro se usado fora do Provider.
selectPatient(patient)	Atualiza estado React E persiste no localStorage('selectedPatient'). Sincroniza sidebar e todas as páginas.

O PatientProvider é inserido no DashboardLayout, envolvendo todas as páginas do /dashboard. Isso garante que ao navegar entre abas (Alertas → Gráficos → Histórico), o paciente selecionado permaneça o mesmo sem nova requisição.

11.2 useActivePatientId — Hook de localStorage

Hook alternativo (use-active-patient.ts) que persiste o ID ativo no localStorage com a chave "p30:activePatientId". Utiliza CustomEvent "p30:activePatientChanged" para sincronizar múltiplas instâncias abertas na mesma aba ou em abas diferentes (via evento "storage" do browser).

Coexistência de dois mecanismos

O PatientContext salva o objeto completo do paciente (id, name, age, condition) enquanto o useActivePatientId salva apenas o ID. O contexto é o mecanismo ativo e principal. O hook de localStorage existe como utilitário de baixo nível e pode ser usado em casos onde o Provider não está disponível.

11.3 useAuth — Hook de Autenticação Client-side

O hook use-auth.ts é um mecanismo de autenticação client-side legado que persiste dados do usuário no localStorage ("cuidamais_user"). Importante: este hook NÃO é o mecanismo de autenticação real da aplicação — a autenticação verdadeira usa JWT via cookie HttpOnly no servidor (lib/auth.ts). O useAuth parece ser um resquício de uma versão anterior e não deve ser confiado para decisões de segurança.

12. Hooks e Utilitários

12.1 useIsMobile

Hook simples que usa `window.matchMedia` para detectar se a tela está abaixo de 768px. Atualiza reativamente via event listener "change" na `MediaQueryList`. Retorna boolean.

12.2 Sistema de Design — `globals.css`

O design system é definido em CSS puro via variáveis customizadas (`@theme inline`) com a biblioteca Tailwind CSS v4. Usa o espaço de cor `oklch` para maior precisão perceptual. Suporta modo escuro via classe `.dark`.

Paleta principal

Token	Valor oklch	Uso
<code>--primary</code>	<code>oklch(0.6 0.14 235)</code>	Azul médico — botões, links, destaques
<code>--success</code>	<code>oklch(0.68 0.14 162)</code>	Verde saúde — estados positivos, glicemia OK
<code>--warning</code>	<code>oklch(0.78 0.15 75)</code>	Âmbar — alertas warning, temperatura
<code>--destructive</code>	<code>oklch(0.6 0.22 25)</code>	Vermelho — alertas críticos, erros, logout
<code>--background</code>	<code>oklch(0.995 0.002 230)</code>	Fundo geral — quase branco com leve tom azul

Fontes

- Corpo de texto: Inter / DM Sans (`--font-sans`)
- Títulos e headings: Plus Jakarta Sans (`--font-display`) com `letter-spacing: -0.02em`

Animações customizadas

- `pulse-ring`: anel pulsante azul para indicadores ativos (2s infinito)
- `float`: levitação suave de elementos decorativos (4s `ease-in-out` infinito)
- `grid-pattern`: padrão de grade sutil para backgrounds de marketing

13. Configuração do Projeto

13.1 TypeScript — tsconfig.json

Opção	Valor / Explicação
target	ES2017 — compatibilidade com Node.js moderno
strict	false — TypeScript em modo permissivo (sem strict null checks)
moduleResolution	bundler — resolução moderna compatível com Next.js/Vite
paths: @/*	./src/* — permite imports como @/lib/auth em vez de ../../lib/auth
paths: @root/*	./* — acesso à raiz do projeto
jsx	react-jsx — JSX automático sem import React necessário
ignoreDeprecations	"6.0" — suprime avisos de APIs depreciadas do TypeScript 6

13.2 Estrutura de Diretórios (src/)

Caminho	Conteúdo
src/app/	Rotas Next.js (App Router) — páginas, layouts, api routes
src/app/api/	Route Handlers da API REST
src/app/dashboard/	Páginas do painel autenticado
src/context/	PatientContext.tsx — contexto global de paciente
src/hooks/	use-active-patient.ts, use-auth.ts, use-mobile.tsx
src/lib/	auth.ts (JWT), prisma.ts (Prisma singleton), toast.ts
src/app/globals.css	Design system — variáveis CSS, Tailwind, animações
prisma/schema.prisma	Schema do banco — modelos User, Patient, Record
prisma/database.db	Arquivo SQLite gerado pelo Prisma